Swiss Summer Student Particle Physics Program

Project Report, July – September 2026

Operating the Mu3e SciFi Detector: Control, Monitoring and Calibration

Giacomo Moretti | Supervisor: Marius Köppel
Mu3e collaboration, SciFi detector group, D-PHYS, ETH Zürich

1 Summary

The Mu3e experiment at the Paul Scherrer Institute (PSI) searches for the charged-lepton-flavour-violating decay $\mu^+ \to e^+e^-e^+$. In the Standard Model this decay proceeds only through a neutrino loop, with a branching ratio of order 10^{-54}, so it is effectively background-free and any observation would be unambiguous evidence of new physics. The current limit, $\mathscr{B} < 10^{-12}$ (SINDRUM, 1988), leaves four orders of magnitude for Mu3e to explore, down to a single-event sensitivity of about 10^{-16}. Reaching it means running a continuous $10^8\,\mu/\mathrm{s}$ beam while suppressing two backgrounds: internal conversion $\mu \to eee\nu\bar{\nu}$, controlled by energy and momentum resolution, and accidental combinatorial background, which grows with rate and is controlled by timing and vertex resolution. The scintillating-fibre (SciFi) detector provides the timing in the central region, with a target resolution below 200 ps.

My project was the SciFi detector's control, monitoring and calibration system, built end to end for the first physics run at PSI in October 2026. At the start, SciFi was operated through a single web page (`Scifi-monitor.js`) that had grown organically: it worked, but it was hard to trust and hard to extend, and it lagged behind the more mature pixel-tracker tooling. I replaced it with a structured set of Python control scripts that drive the hardware and a new custom web page that a shift operator can rely on, both running inside the MIDAS data-acquisition framework and communicating through its online database (ODB).

The system was built in layers: installation and bring-up at PSI; the operator interface; the power chain (low voltage, high voltage, front-end boards); a configuration and hardware-test hub; the two SiPM calibrations (threshold and breakdown-voltage scans); and the saving of calibration results to three destinations – the live ODB, a local per-board cache, and the permanent calibration database (CDB). By the end of the project the detector – low voltage, high voltage, front-end boards and readout – is installed and working on the experiment back-end, the calibration result-saving path is fully implemented and wired, and the shared control-and-monitoring code library now covers SciFi as well as the pixel detector. The main item left is a full end-to-end test on the experiment, including a real calibration run. Alongside the software I also took part in the on-site detector work – cabling, power distribution, assembly, repairs and general service work across the full Mu3e detector – and in the collaboration's routine of weekly reports and coordination meetings.

2 Project Details

2.1 State-of-the-art and Objectives

Physics context. Charged lepton flavour violation (cLFV) has never been observed. Lepton flavour is violated in the neutral sector – neutrinos oscillate – but in the charged sector the Standard Model forbids it to any measurable level: $\mu^+ \to e^+e^-e^+$ is only possible through a neutrino loop and is predicted at $\mathscr{B} \sim 10^{-54}$ [1]. Many extensions of the Standard Model raise this rate to within experimental reach, which makes $\mu \to eee$ one of the "golden" muon channels, alongside $\mu \to e\gamma$ and μ–e conversion. The strongest existing bound, $\mathscr{B} < 1.0 \times 10^{-12}$ at 90% C.L., was set by SINDRUM in 1988 [2]; Mu3e aims to improve on it by four orders of magnitude, to a single-event sensitivity of about 10^{-16}, using the most intense continuous surface-muon beam in the world at PSI [1].

Detector and data acquisition. Mu3e reconstructs the three-electron final state with ultra-thin high-voltage monolithic active pixel sensors and measures time with two scintillator systems: the SciFi detector in the central region (target resolution $< 200\,\text{ps}$) and scintillating tiles in the recurl stations ($< 50\,\text{ps}$) [3, 4]. The SciFi detector is arranged in two rings, upstream and downstream, of twelve modules each; its 96 MuTRiG readout ASICs (32 channels each, 3072 channels in total) are served by twelve front-end boards and read out through the MIDAS framework [5], whose online database holds all configuration and slow-control data and whose web server hosts custom operator pages. A single module key – side, ring position and readout end – is used consistently across the ODB, the detector geometry and the calibration [6].

Starting point and where work was needed. The pixel subdetector already had a mature MIDAS integration: dedicated operator pages and a structured library of Python control scripts. SciFi integration was less advanced. It relied on a single monitoring page that had accreted features over time, with control logic scattered across disconnected scripts and no interface a shift operator could depend on during a run. With the first physics run scheduled for October 2026, SciFi needed a coherent operator interface, a documented and reproducible calibration procedure whose results reach the CDB, and a control code base shared with – rather than duplicating – the pixel tooling.

Contribution. A dependable control and monitoring system is a prerequisite for taking SciFi data at all, and unifying the SciFi and pixel software lowers the long-term cost of operating the experiment. Both directly support Mu3e Phase I data taking.

2.2 Methodology

Environment and existing inputs. All work targets the MIDAS DAQ: configuration and state live in the ODB, operator interfaces are custom HTML/JavaScript pages served by the MIDAS web server, and device logic is Python running on the front-end machine. No new physics data was collected – the project builds on what the DAQ already exposes: the ODB slow-control and configuration tree, the existing pixel control library and the diverged SciFi scripts, and the raw per-channel data produced by MuTRiG threshold and current–voltage scans and by the hardware tests. The fixed hardware layout (two rings, twelve modules and front-end boards, 96 ASICs, 24 high-voltage channels) is the reference against which ODB contents were checked. Development used a local test ODB, with checks against real hardware on the experiment backend.

Existing methods vs. what had to be developed. The pixel subdetector's pages and control

scripts were the existing method and were reused as a template: the same web-server/ODB binding pattern and the same "page-first" workflow – mock up the layout, agree the control granularity with the SciFi experts, then write the Python backend. What did not exist and had to be built were the entire SciFi page and its ODB bindings (the pixel page is not reusable given the different detector topology); SciFi coverage in the shared control library, factored around subdetector-agnostic abstractions for the front-end board, ASIC and channel; a configuration tab that doubles as the detector's hardware-test hub; and the two SiPM calibrations on a common lifecycle with a working path to permanent storage.

The interface. The new page separates a read-only Geometry view – a cumulative drill-down from super-module to module to the 32-channel ASIC grid, drawn as mirrored upstream and downstream rings – from the editable Power, Readout and Calibration tabs. Every editable value is written to the ODB, from where a front-end process applies it to the hardware; safety logic stays in Python and is never duplicated in JavaScript. No action is ever hard-blocked: controls stay live and the page warns rather than refusing. The largest remaining hardcoded element, the 32-cell high-voltage board-to-module map, is flagged for a later move into Python/ODB.

Configuration and hardware tests. The configuration tab grew beyond parameter editing into a general hardware-test hub – the former Readout tab folds into it entirely. It walks the front-end board → module → chip → channel tree, writing to a dedicated ODB subtree, and runs three tests: PLL (each board must recover the clock from its data link – the recovered-clock delay histogram is flat when locked, sharply peaked when not), PRBS (link integrity) and TDC (per-channel test pulse).

Calibration. Two per-board scans share one `prepare` / `execute` / `finalise` lifecycle. The threshold scan walks the discriminator threshold on every channel and reads the rate curve to locate the 0.5, 1.5 and 2.5 photo-electron crossings; its output is loaded as the operating threshold. The breakdown-voltage scan takes the derivative of each SiPM's current–voltage curve and fits a Gaussian to the peak, whose mean gives the breakdown voltage and hence the operating overvoltage – a physics constant, not just a diagnostic. Each scan runs independently per front-end board, so any subset of the detector can be re-calibrated on its own. A run is launched from the page, which writes the request to the ODB; the MIDAS PySequencer picks up the corresponding script, which opens its own MIDAS connection (with an explicit disconnect on exit) and executes the lifecycle.

Saving. Results go to three destinations, one for each question an operator asks: the live ODB (*"what is the calibration right now?"*, overwritten each scan), a local on-disk cache of the most recent calibration per board (*"what did we last calibrate, for day-to-day operations?"*), and the permanent CDB, indexed by name and run number (*"what was the calibration on a given run?"*). The on-disk format is a small ROOT file with separate trees for metadata, raw scan, results, conditions and channel masks. This path was reconciled with a colleague's parallel work on the same code.

Verification. Each branch – the power bring-up sequences, the hardware tests, the calibration scans – was built and checked on its own before being merged into the combined system, first against the local test ODB and then against real hardware on the experiment back-end.

Hardware and on-site work. The software was not developed apart from the detector. It was written inside the collaboration's on-site campaigns at PSI, where I also worked directly on the hardware – cabling, power distribution, assembly and repairs – so the power and readout chains the page has to represent are largely the ones I helped bring up by hand, and the interface design followed from that. The project also ran on the collaboration's normal working rhythm through-

out: weekly progress reports, subdetector and DAQ meetings, and continuous coordination with the DAQ team, who own the shared code the SciFi tooling now builds on.

2.3 Results

The three objectives – a dependable operator interface, a reproducible calibration procedure feeding the CDB, and a control code base shared with the pixel detector – were all met, and everything that had to be ready for the October run is in place.

Operator interface. The new page is complete and replaces `Scifi-monitor.js` in operation. The Power, Readout and Calibration tabs are bound to the ODB and verified against the test ODB; the read-only Geometry view is fully functional; and readout-status monitoring (data rate per board, run state, buffer occupancy) is integrated, so that "power all green" and "data flowing" are presented as the two distinct conditions they are. Where the operator previously had an unlabelled static diagram, there is now one coherent, live view of the detector.

Configuration and hardware tests. The configuration tab reached beyond the planned parameter editor and became the detector's general hardware-test entry point, absorbing the former Readout tab. The PLL, PRBS and TDC tests all run from it and return one verdict per board or per link.

Calibration and saving. Both scans run per front-end board and produce the values actually loaded for operation (discriminator threshold and SiPM overvoltage). The result-saving path – the one piece that strictly had to be ready for data taking – is fully implemented and wired to all three destinations (live ODB, local per-board cache, permanent CDB), with the on-disk ROOT layout finalised.

Shared code and status on the experiment. The common control library now covers SciFi as well as the pixel detector, so the duplication that motivated the project is gone. On the experiment back-end, low voltage, high voltage, front-end boards and readout are up and were operated through the new page during the installation week at PSI. The SciFi cabling and power distribution are wired and under management, and the assembly, repair and service work done during the on-site periods – part of it across the full Mu3e detector – leaves the apparatus in a running state.

Consistency with expectations and future developments. The delivered scope matches what was planned with the SciFi group at the start of the summer; the one notable deviation – the configuration tab growing into a full test hub – was in the useful direction. The principal open item is a single end-to-end workflow test on the experiment, page through to all three save destinations, including a real calibration run rather than test-ODB checks. Smaller follow-ups are moving the hardcoded high-voltage board map into Python/ODB, extending the online monitoring histograms, and wiring SciFi's own `CalibrationFactory` to a real entry point so the scripts no longer need their simplified direct-construction path.

2.4 Your contributions as an S3P3 student

I worked within the Mu3e collaboration, SciFi detector group at ETH Zürich, in coordination with the pixel DAQ developers, who own the shared code and whose tooling served as the reference. The control, monitoring and calibration system described in the previous sections is my own work, built over the summer:

- I designed and implemented the new custom page end to end – the read-only Geometry view and the editable Power, Readout and Calibration tabs, with their ODB bindings and Python backend – and took the design decisions that shape it, in particular keeping all safety logic in Python and having the page warn rather than block.
- I turned the configuration tab into the hardware-test hub and implemented the PLL, PRBS and TDC test flows.
- I implemented the threshold and breakdown-voltage calibrations on their shared lifecycle and the full result-saving path to the ODB, the local cache and the CDB, including the ROOT layout, reconciling it with a colleague's parallel branch.
- I extended the common control library to cover SciFi, factoring the front-end-board, ASIC and channel abstractions it is built on.

Hardware, service and collaboration. The software was one half of the work; the other was hands-on. During the on-site periods at PSI I did SciFi cabling and power-distribution management, mechanical assembly, and repairs, and I designed small hardware components. A large part of my time went into general service work on the full Mu3e detector rather than SciFi alone, including direct contributions to other subdetectors – assembly, repair and maintenance tasks across the apparatus. On the organisational side I reported progress weekly, took part in the subdetector and DAQ meetings, and coordinated continuously with the DAQ team, whose shared code base the SciFi tooling was built to extend rather than duplicate. This close contact with the collaboration shaped most of the design decisions in the sections above.

References

[1] Mu3e Collaboration, "Technical design of the Phase I Mu3e experiment", arXiv:2009.11690 (2021).

[2] U. Bellgardt *et al.* (SINDRUM Collaboration), "Search for the decay $\mu^+ \to e^+e^+e^-$", Nucl. Phys. B **299** (1988) 1.

[3] A. Bravar *et al.*, "Development of the Scintillating Fibre Timing Detector for the Mu3e Experiment", arXiv:2208.09906 (2022).

[4] L. Vigani *et al.*, "The Mu3e detector", JINST **17** (2022) C05024.

[5] M. Köppel (Mu3e Collaboration), "Mu3e Integration Run 2021", arXiv:2203.07855 (2022).

[6] Y. Wang (Mu3e Collaboration), "The DAQ system of the Mu3e SciFi detector", SPS/APS Joint Annual Meeting (2023).